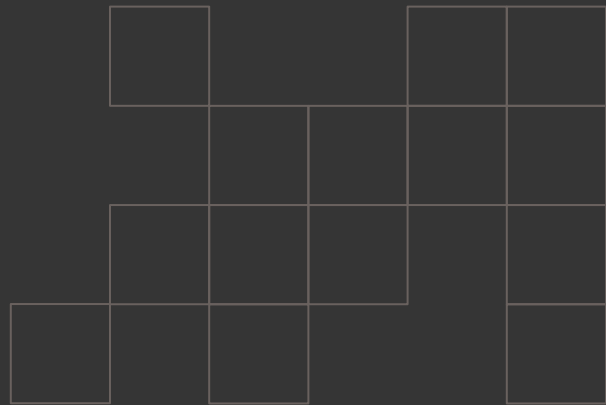


Spencer Dearman



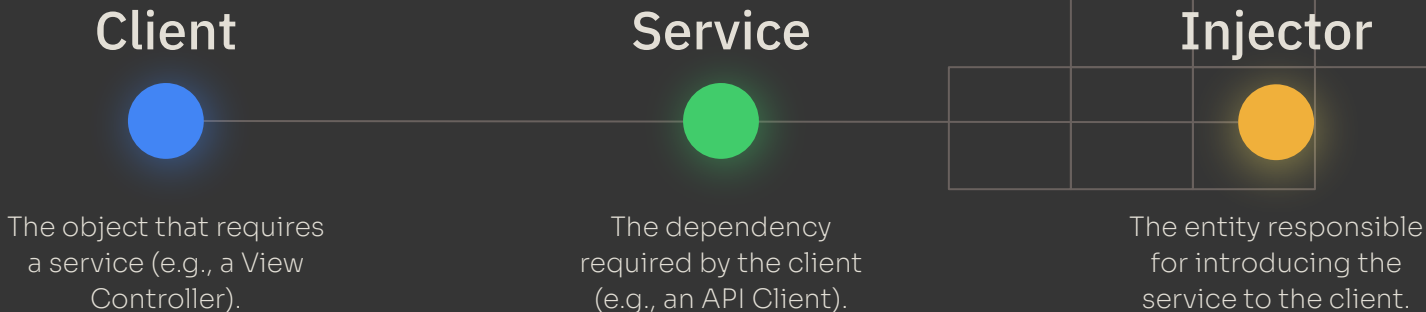
Swinject

A Swift Dependency-Injection Framework



What is Dependency Injection?

Dependency Injection is a design pattern where an object's dependencies are provided externally rather than created internally.



Inversion of Control: Shifting the responsibility of object creation and lifecycle management away from the functional code.

DI Example

```
class UserProfileViewModel {  
    // Hidden dependencies using Singletons  
    private let api = NetworkManager.shared  
    private let database = LocalStorage.shared  
    private let logger = Analytics.shared  
  
    func loadUser() {  
        api.fetchUser { user in  
            self.database.save(user)  
            self.logger.log("User Profile Loaded")  
        }  
    }  
}
```

Key Issues

- Test Fragility
- Rigidity
- Hidden Complexity

```
// Contracts  
protocol UserNetworkService {  
    func fetchUser(completion: @escaping (User) -> Void)  
}  
  
protocol UserStorageService {  
    func save(_ user: User)  
}  
  
class UserProfileViewModel {  
    private let api: UserNetworkService  
    private let database: UserStorageService  
  
    // Inject the dependencies via Initializer  
    init(api: UserNetworkService, database: UserStorageService) {  
        self.api = api  
        self.database = database  
    }  
  
    func loadUser() {  
        api.fetchUser { user in  
            self.database.save(user)  
        }  
    }  
}
```

Why is DI Important in iOS?

Testing

Easily swap a **LiveNetworkService** for a **MockNetworkService** during unit tests.

Maintainability

Changes in a dependency's implementation **do not require changes** in the client class.

Separation of Concerns

Functional classes **focus on logic**, while the Dependency Injection Container focuses on **object construction**.

Parallel Development

Teams can work on different pieces of code simultaneously by agreeing on protocols.

Types of Injection

Initializer Injection

Preferred Method

Passing dependencies through the init method. Ensures the object is never in an invalid state.

Property Injection

Assigning dependencies after the object is initialized. Useful for objects created by Storyboards or XIBs.

Method Injection

Passing a dependency only when a specific method is called.

How does Swinject work?

Swinject Structure

1. Container

A central registry that maps abstractions (Protocols) to concrete implementations.

2. Registration

Defining how dependencies are created.

- Uses a register method to store a factory closure.
- Identifies dependencies by their service type and optional name.

3. Resolution

Retrieving dependencies from the container.

- Uses a **resolve** method to instantiate objects.
- Automatically resolves sub-dependencies in the object graph.

```
let container = Container()

// Registration
container.register(DataService.self) { _ in
    NetworkService()
}

// Resolution
let service = container.resolve(DataService.self)
```

Swinject PR

GitHub Link: <https://github.com/Swinject/Swinject/pull/591>

Make maxResolutionDepth configurable in Container #591

 Open `spencerdearman` wants to merge 1 commit into `Swinject:master` from `spencerdearman:master` 

 Conversation 0  Commits 1  Checks 0  Files changed 2



spencerdearman commented 2 minutes ago 

This PR changes the `maxResolutionDepth` property from a hardcoded fileprivate constant to a configurable stored property within the `Container` class.

Motivation

In the current implementation, the recursion limit for dependency resolution is fixed at 200. While this is sufficient for most use cases, complex dependency graphs in enterprise-scale architectures or machine-generated code can exceed this limit. When reached, the container triggers a `fatalError` that developers currently cannot circumvent without modifying the framework source. Making this limit configurable provides the necessary flexibility for large-scale applications while maintaining a safe default.

Swinject Code Walkthrough

Container:

<https://github.com/Swinject/Swinject/blob/master/Sources/Container.swift>

Resolver:

<https://github.com/Swinject/Swinject/blob/master/Sources/Resolver.swift>

Swinject Demo

